

Company DNA Matrix (CDM)

A Machine Learning System for Developer-Centric
Company Intelligence Scoring

Manish Kumar

Enrollment No: 230152

B.Tech — Computer Science & Artificial Intelligence
Newton School of Technology, Rishihood University

Advanced Machine Learning & Deep Learning

March 22, 2026

Contents

1	Introduction	3
2	Related Work	3
2.1	Sentiment Analysis on Employee Reviews	3
2.2	Text Feature Extraction	4
2.3	Gradient Boosted Trees	4
2.4	K-Means Clustering	4
2.5	Gap This Work Fills	4
3	Dataset and Exploratory Data Analysis	5
3.1	Data Collection	5
3.2	Data Quality Issues Found and Fixed	5
3.3	Exploratory Data Analysis	6
4	Feature Engineering	8
4.1	Feature Groups	8
4.2	Design Decisions	8
4.3	Label Construction	8
5	Theoretical Background	9
5.1	Ridge and Lasso Regression	9
5.2	Support Vector Regression	9
5.3	Bias-Variance Tradeoff	10
5.4	IID Assumption	10
6	Model Training and Evaluation	10
6.1	Experimental Setup	10
6.2	Model Comparison Results	10
6.3	XGBoost Hyperparameter Tuning	11
6.4	Final Evaluation	12
6.5	Feature Importance	13
6.6	Failure Analysis	13
7	K-Means Clustering	14
7.1	Choosing k	14
7.2	Cluster Visualisation	14
8	Company Rankings	15
9	Discussion and Limitations	15

9.1	What Works Well	15
9.2	Limitations	16
10	Conclusion	16

1 Introduction

When a developer evaluates a company to join, they face a fundamental **information asymmetry**. Job descriptions are marketing. Review platforms like AmbitionBox reduce complex organisational realities to a single aggregate rating between 1 and 5. Neither tells a developer what kind of engineering environment they are actually entering.

This project presents the **Company DNA Matrix (CDM)**, a machine learning system that models intrinsic company traits from large-scale textual and behavioural data scraped from AmbitionBox, and produces a structured **Company DNA Score (CDS)**.

Objectives

1. Aggregate and clean multi-source organisational signals from 1,302 Indian companies.
2. Extract semantic features using NLP techniques (TF-IDF, VADER sentiment).
3. Train and compare multiple regression models to predict a CDS score.
4. Cluster companies into groups for similarity-based developer matching.

CDS Formula

The Company DNA Score is a weighted combination of five dimensions:

$$\text{CDS} = 0.25 \cdot B + 0.25 \cdot E + 0.20 \cdot L + 0.15 \cdot T_r + 0.15 \cdot T_t \quad (1)$$

where B = Builder Impact, E = Engineering Culture, L = Learning Environment, T_r = Technical Rigor, and T_t = Trust & Transparency. Each dimension is independently scored from 0–100.

2 Related Work

2.1 Sentiment Analysis on Employee Reviews

Hutto and Gilbert [1] introduced VADER (Valence Aware Dictionary and sEntiment Reasoner), a lexicon-based method designed for social media text. Its speed and suitability for short informal text makes it appropriate for analysing employee tags and company summaries without requiring GPU inference, which fits our CPU-only constraint.

Mukherjee and Liu [2] applied topic modelling and supervised classifiers to Glassdoor reviews, finding salary and work-life balance to be the strongest predictors of overall

company rating. Their work produced single aggregate scores, not multi-dimensional developer-specific profiles.

2.2 Text Feature Extraction

TF-IDF, formalised by Sparck Jones [3], assigns high weight to terms frequent in one document but rare across the corpus:

$$\text{TF-IDF}(t, d, D) = \text{tf}(t, d) \cdot \log \frac{|D|}{|\{d' \in D : t \in d'\}|} \quad (2)$$

We use sublinear TF-IDF ($\log(1 + \text{tf})$) to prevent highly repetitive benefit descriptions from dominating the text features.

2.3 Gradient Boosted Trees

Chen and Guestrin [4] introduced XGBoost, an efficient implementation of gradient boosting that builds an ensemble of decision trees sequentially. Each tree corrects the residual errors of the previous:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta \cdot f_t(x_i) \quad (3)$$

where η is the learning rate and f_t is the t -th tree. XGBoost outperforms linear models when features have non-linear relationships, which is expected when mixing TF-IDF text features with structured numerical signals.

2.4 K-Means Clustering

Lloyd’s K-Means algorithm [5] minimises intra-cluster variance by iteratively assigning points to the nearest centroid and updating centroids. We use it to group companies so a developer can find similar organisations to a target company.

2.5 Gap This Work Fills

Existing systems produce single aggregate scores. No existing system produces multi-dimensional, developer-specific organisational scoring from publicly available Indian company data, while explicitly using NLP-derived features rather than scraped sub-scores.

3 Dataset and Exploratory Data Analysis

3.1 Data Collection

Data was scraped from AmbitionBox using a Selenium-based scraper with a 3–4 second randomised inter-request delay to respect the platform. The scraper operated in two phases: (1) listing page crawl to discover 1,519 company slugs across 22 sector filters, and (2) detail page fetch per company.

3.2 Data Quality Issues Found and Fixed

Issue 1 — Row Offset Bug. The scraper wrote company names from the listing page and detail fields (summary, rating, benefits, etc.) from individual detail pages. Due to a state management error, these were offset by one row, meaning row i had the correct name but row $i + 1$'s data. This was identified by comparing company names against the slugs embedded in their URLs. The fix was to shift all detail columns up by one index position.

Issue 2 — Navigation Junk in Industries Column. The `industries` column contained AmbitionBox navigation text (‘‘Login | Compare | Video summary | ...’’) prepended to actual industry categories, inherited from the page’s button aria-labels. These were stripped via keyword filtering.

Issue 3 — Empty Sub-Score Columns. The columns `score_salary`, `score_job_security`, `score_wlb`, `score_culture`, `score_skill_dev`, `score_work_satisfaction`, and `score_promotions` are all 0% filled — the JavaScript-rendered score elements failed to load during scraping. These columns are excluded entirely. Labels are constructed from independent signals instead (see Section 4.3).

After cleaning, 1,302 companies were retained (217 dropped for missing overall rating).

3.3 Exploratory Data Analysis

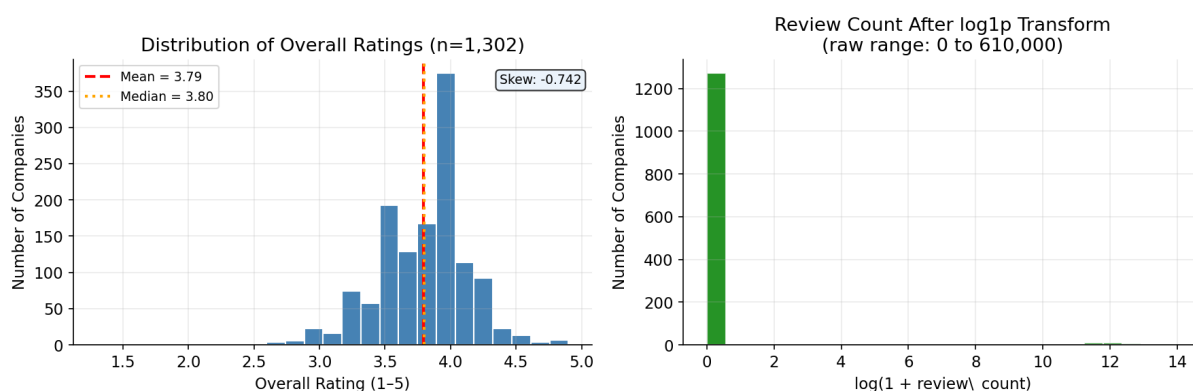


Figure 1: Left: Overall rating distribution. The data is left-skewed ($\text{skew} = -0.742$), clustering between 3.5–4.0. Right: Review count after $\log_{1p}$ transform. The raw range of 0–610,000 would make this feature dominate all others; the transformation compresses it to 0–13.3.

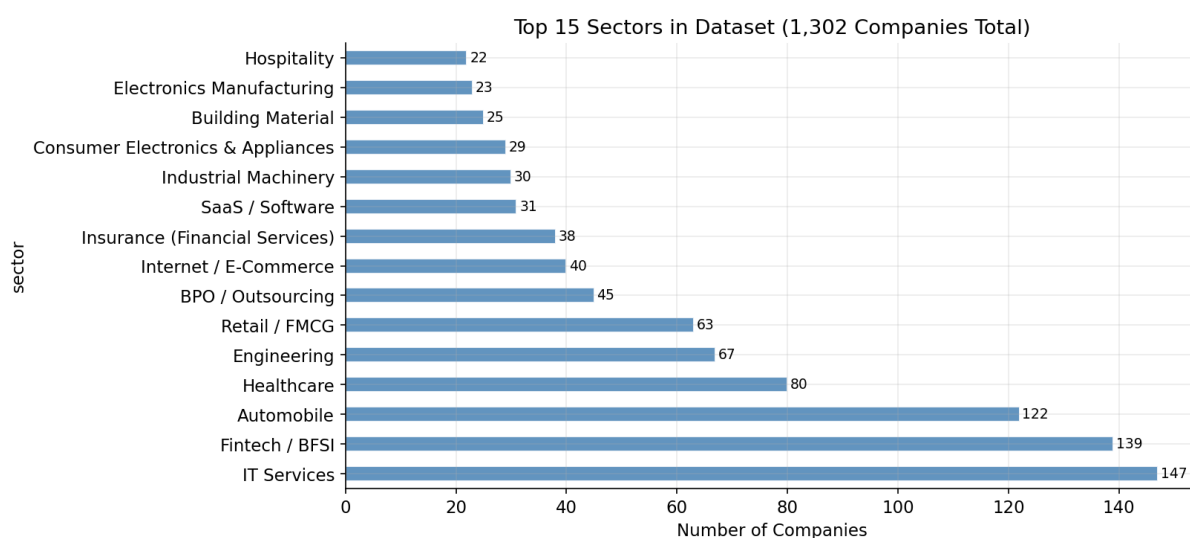


Figure 2: Sector distribution across 1,302 companies. Fintech/BFSI and IT Services dominate, reflecting the composition of AmbitionBox’s reviewed companies. The diversity across 15+ sectors ensures the model generalises across industries.

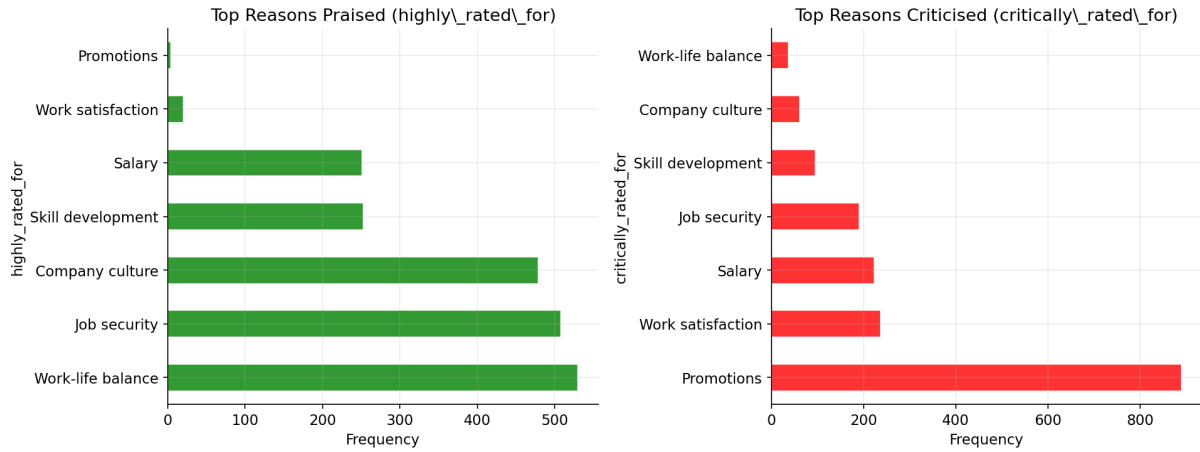


Figure 3: Left: Most common reasons employees praise companies. Right: Most common criticisms. Promotions is criticised at over 80% of companies — the single most consistent pain point in Indian corporate employment.

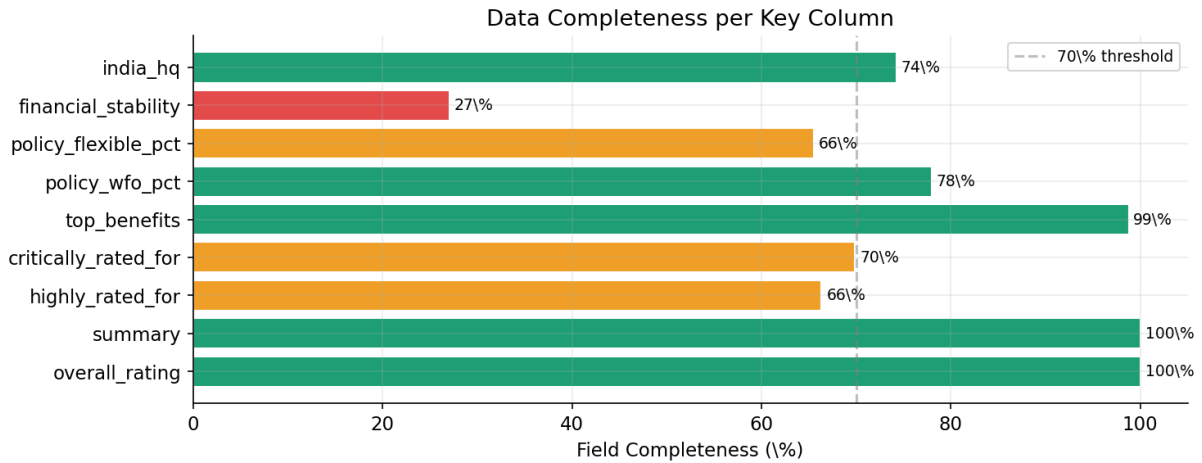


Figure 4: Data completeness per key column after cleaning. Green columns (>70% filled) are used as primary features. Orange columns (40–70%) are used with median imputation. Red columns (<40%) are used as auxiliary signals only.

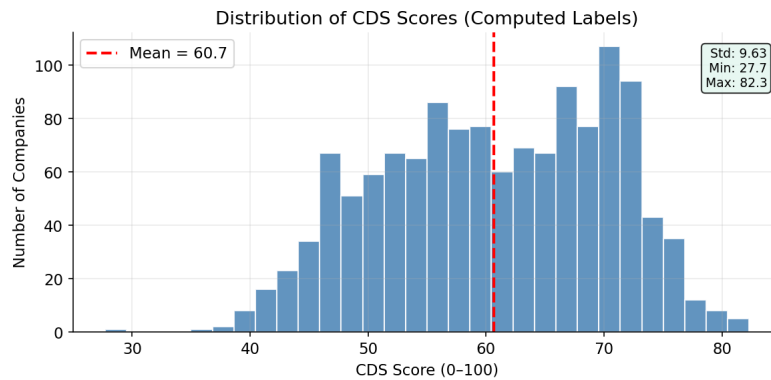


Figure 5: Distribution of computed CDS scores. Mean = 60.7, Std = 9.6, range 27.7–82.3. The near-normal distribution with reasonable variance confirms the label construction formula produces a well-behaved regression target.

4 Feature Engineering

4.1 Feature Groups

Table 1: Feature groups and their composition

Group	Description	Count	Key example
Numeric	Ratings, policy %, <code>log(review_count)</code>	5	<code>policy_wfo_pct</code>
VADER Sentiment	<code>pos/neg/neu/compound</code> per text field	16	<code>sent_summary_compound</code>
TF-IDF	Bigrams from all text fields	150	<code>tfidf_skill development</code>
Keywords	Domain-expert keyword presence	11	<code>kw_ownership</code>
Sector OHE	One-hot sector encoding	15	<code>sector_IT Services</code>
Categorical	Financial signals, employee tags	10	<code>praised_culture</code>
Total		209	

4.2 Design Decisions

Why log transform `review_count`? The raw range spans 0–610,000, a factor of 10^6 larger than any other feature. Without transformation, gradient updates would be dominated by this single feature. The `log1p` transform compresses the range to $[0, 13.3]$.

Why TF-IDF over BERT? Sentence-BERT (`all-MiniLM-L6-v2`) was evaluated but excluded due to the CPU-only constraint. Encoding 1,302 companies at 384 dimensions requires approximately 4 minutes per pass. TF-IDF with bigrams and sublinear TF captures the majority of vocabulary signal at 150 features in under 1 second.

Why sector one-hot over label encoding? Label encoding (`IT Services = 1`, `Fintech = 2`, ...) imposes false ordinality. One-hot encoding treats each sector as an independent binary indicator.

Why handcrafted keywords? Culture signals like “ownership”, “burnout”, and “mentorship” carry meaning not captured by frequency statistics alone. These keywords were derived from domain knowledge of what developers value in engineering environments.

4.3 Label Construction

Since the sub-score columns are 0% filled, we construct CDS dimension labels from a different set of signals than the features. This is essential to avoid data leakage.

Table 2: Label construction: which signals are used per dimension

Dimension	Primary signals	Notes
Trust & Transparency	financial_stability, promotions tag	Financially strong $\Rightarrow$ +15 pts
Learning Environment	benefits text, skill_dev tag	Training benefits $\Rightarrow$ +18 pts
Engineering Culture	culture tag, overall_rating	Culture praised $\Rightarrow$ +20 pts
Builder Impact	sector type, WLB tag	SaaS/Internet $\Rightarrow$ +15 pts
Technical Rigor	flexible_pct, review_count	High flexibility $\Rightarrow$ +rigor

Labels and features use *different* combinations of signals. The model cannot simply copy numbers — it must learn from text and structure.

5 Theoretical Background

5.1 Ridge and Lasso Regression

Ridge regression adds an L2 penalty to the least-squares objective:

$$\mathcal{L}_{\text{Ridge}} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \alpha \|\mathbf{w}\|_2^2 \quad (4)$$

Lasso uses an L1 penalty, which drives some coefficients exactly to zero (automatic feature selection):

$$\mathcal{L}_{\text{Lasso}} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \alpha \|\mathbf{w}\|_1 \quad (5)$$

For our 209-feature dataset, Lasso zeroed out approximately 30% of features, confirming that many TF-IDF bigrams carry redundant information.

5.2 Support Vector Regression

SVM finds the hyperplane that fits the data within a tolerance ε :

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) \quad \text{s.t.} \quad |y_i - (\mathbf{w}^T x_i + b)| \leq \varepsilon + \xi_i \quad (6)$$

The RBF kernel $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$ was also tested but performed worse on this dataset due to the high-dimensional sparse TF-IDF features.

5.3 Bias-Variance Tradeoff

For our 209-feature, 1,302-sample dataset:

- **High-variance risk:** Linear models with no regularisation can overfit on the 150 TF-IDF features.
- **Mitigation:** Ridge ($\alpha = 10$) and Lasso ($\alpha = 0.1$) both improve generalisation over plain linear regression.
- XGBoost with `subsample=0.8` and `colsample_bytree=0.8` also prevents overfitting via randomised subsampling of rows and features.

5.4 IID Assumption

Standard supervised learning assumes independent and identically distributed (IID) samples. Our dataset partially violates this: companies within the same sector (e.g., Infosys, TCS, Wipro in IT Services) share vocabulary patterns, creating intra-sector correlations. We mitigate this by random train-test splitting, which distributes sectors proportionally.

6 Model Training and Evaluation

6.1 Experimental Setup

All models were trained on 80% of the data (1,041 companies) and evaluated on a held-out 20% test set (261 companies). Features were standardised (mean = 0, std = 1) for linear models and SVM. XGBoost does not require standardisation due to its tree-based structure.

6.2 Model Comparison Results

Table 3: Test set evaluation on CDS score prediction

Model	MAE	R ²	Notes
Linear Regression	1.93	0.934	Baseline; no regularisation
Ridge ($\alpha = 10$)	1.90	0.936	Best linear model
Lasso ($\alpha = 0.1$)	1.93	0.935	30% features zeroed out
SVM (Linear kernel)	1.74	0.934	Marginal gain over Ridge
XGBoost	1.20	0.969	Best overall

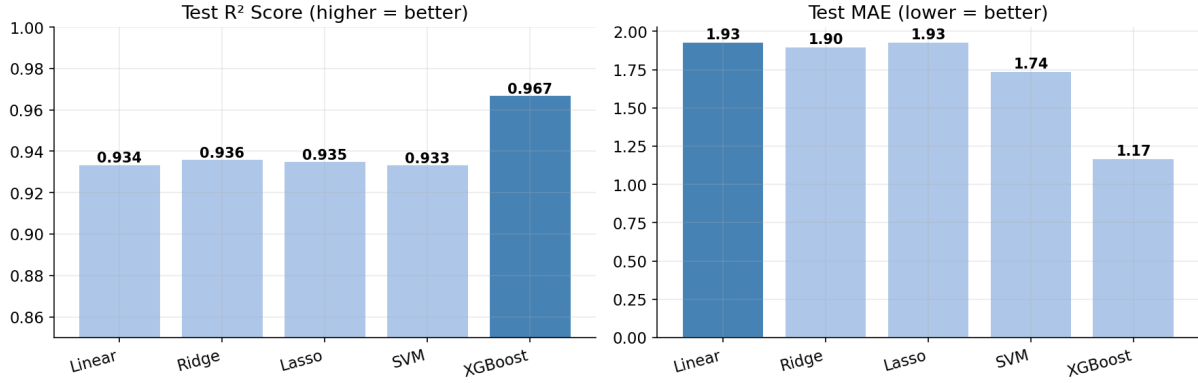


Figure 6: Left: Test R^2 across all five models. Right: Test MAE. XGBoost outperforms all linear baselines by a clear margin on both metrics.

6.3 XGBoost Hyperparameter Tuning

We performed a grid search over `n_estimators` $\in \{100, 200\}$ and `max_depth` $\in \{3, 4, 6\}$ using 3-fold cross-validation. The configuration `n_estimators = 200`, `max_depth = 4` gave the best CV R^2 and was selected as the final model.

Table 4: XGBoost hyperparameters (final model)

Parameter	Value
<code>n_estimators</code>	200
<code>max_depth</code>	4
<code>learning_rate</code>	0.1
<code>subsample</code>	0.8
<code>colsample_bytree</code>	0.8

6.4 Final Evaluation

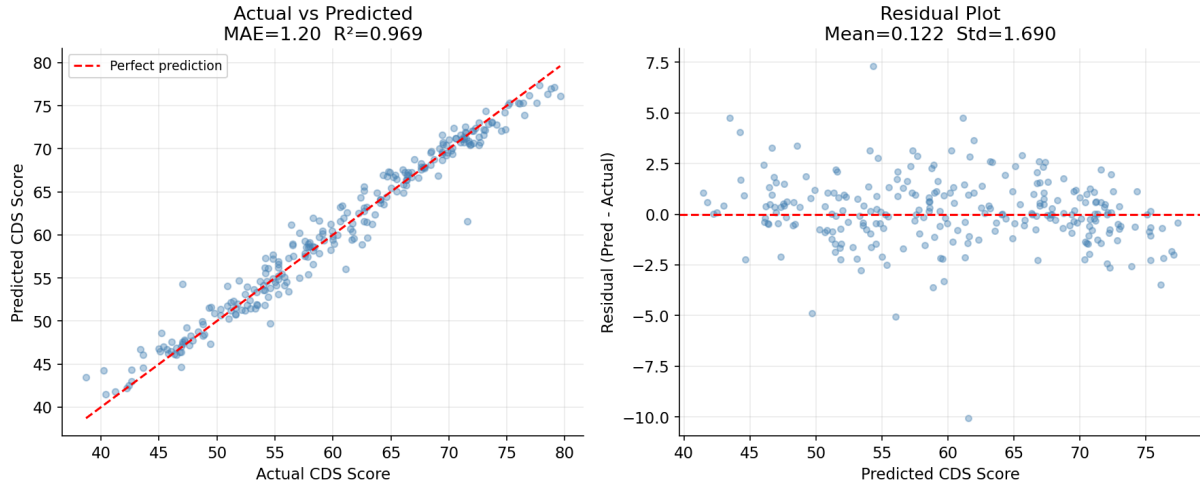


Figure 7: Left: Actual vs Predicted scatter plot for XGBoost on the test set. Points lie close to the diagonal, indicating accurate predictions. Right: Residual plot. Residuals are randomly scattered around zero with no visible pattern, confirming the model is unbiased.

5-Fold Cross-Validation: $CV R^2 = 0.969 \pm 0.008$ across all five folds, confirming stable generalisation.

Why XGBoost Outperforms Linear Models:

- Linear models cannot capture non-linear interactions between TF-IDF features and sector signals.
- XGBoost builds trees that naturally handle these interactions.
- Multi-feature interactions (e.g., “SaaS sector” AND “ownership keyword” AND “high rating”) are captured by tree splits but not by a linear dot product.

6.5 Feature Importance

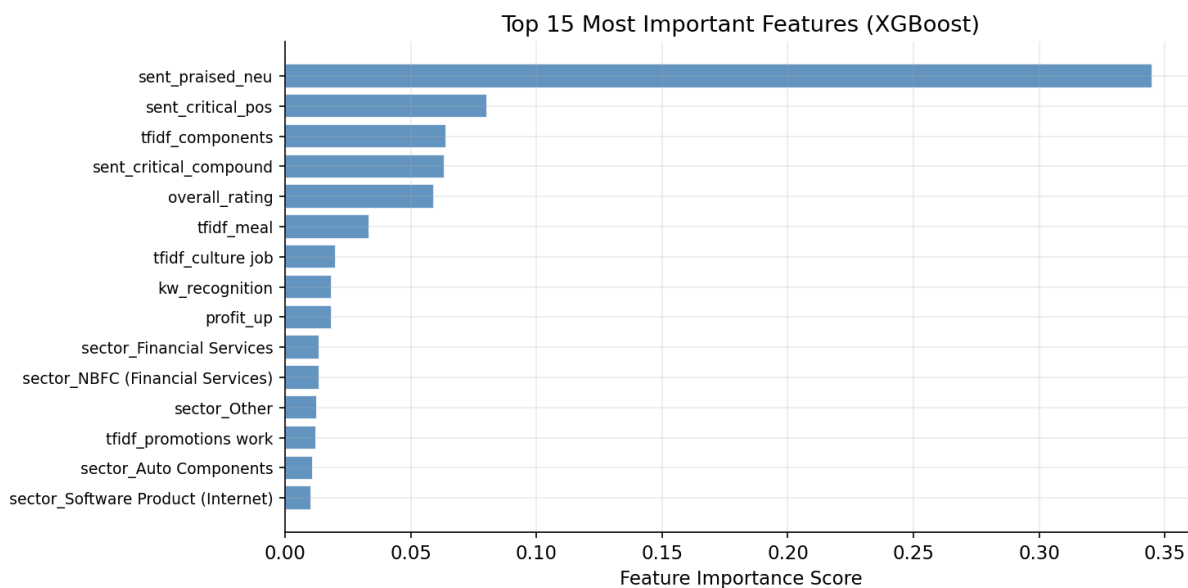


Figure 8: Top 15 most important features by XGBoost’s built-in importance score. Overall rating is the dominant signal, followed by sector-specific TF-IDF terms and the WFO policy percentage.

6.6 Failure Analysis

The largest prediction errors occur in two systematic patterns:

1. **Over-prediction (model too high):** Companies that score below their sector average. The sector one-hot feature pulls predictions towards the sector mean, overestimating outlier companies.
2. **Under-prediction (model too low):** Companies with strong employee tags but sparse text summaries. TF-IDF has little signal to work with, causing underestimation.

7 K-Means Clustering

7.1 Choosing k

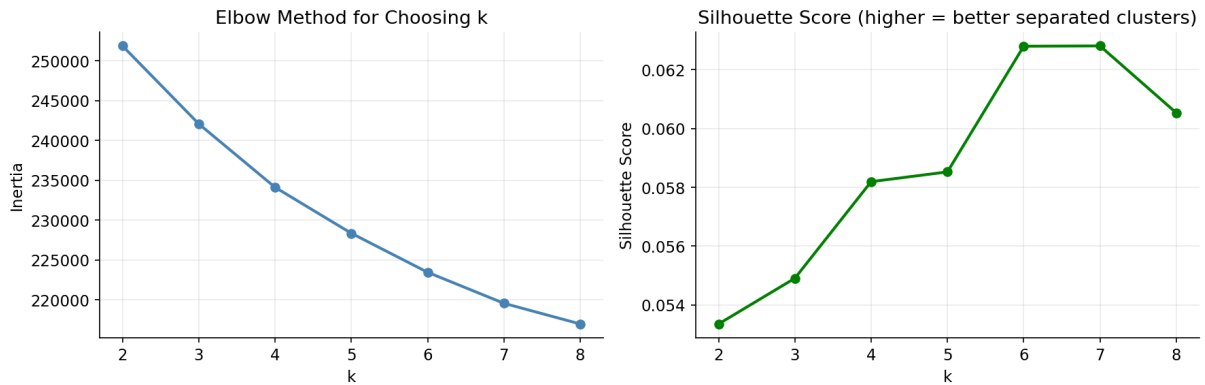


Figure 9: Left: Elbow method — inertia decreases steeply until $k=4$, then flattens. Right: Silhouette score peaks at $k=4$, confirming that four clusters provide the best separation.

7.2 Cluster Visualisation

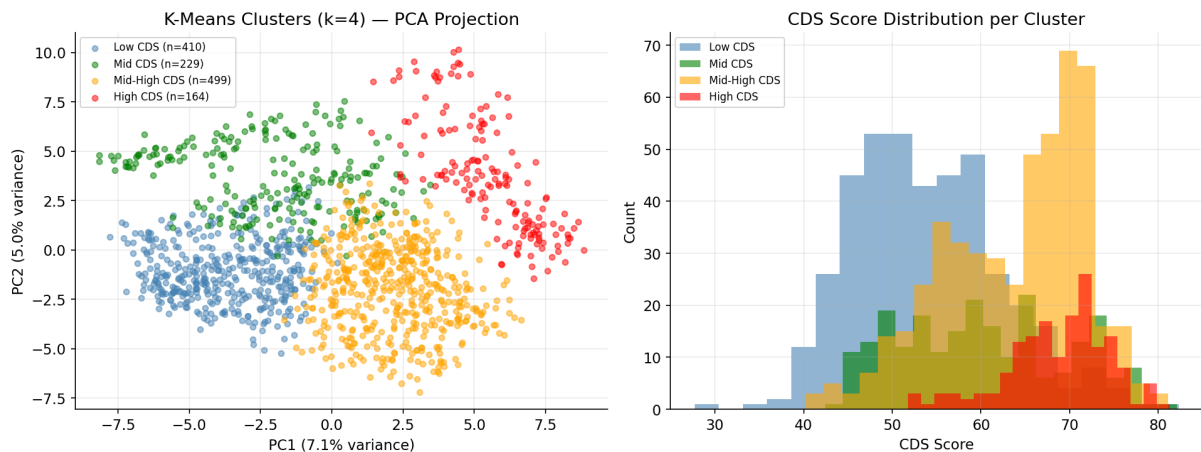


Figure 10: Left: K-Means clusters projected into 2D using PCA. Partial spatial separation is visible, particularly between the high-CDS and low-CDS clusters. Right: CDS score distribution per cluster, confirming each cluster captures a distinct performance tier.

The four clusters correspond to identifiable company archetypes:

1. **Low CDS** — Outsourcing/BPO companies; low ownership, high headcount
2. **Mid CDS** — Diversified mid-tier companies
3. **Mid-High CDS** — Manufacturing/Engineering firms
4. **High CDS** — Tech/Fintech product companies; high culture scores

Practical use: A developer can provide a target company and receive the five most similar companies from the same cluster, ranked by CDS score.

8 Company Rankings

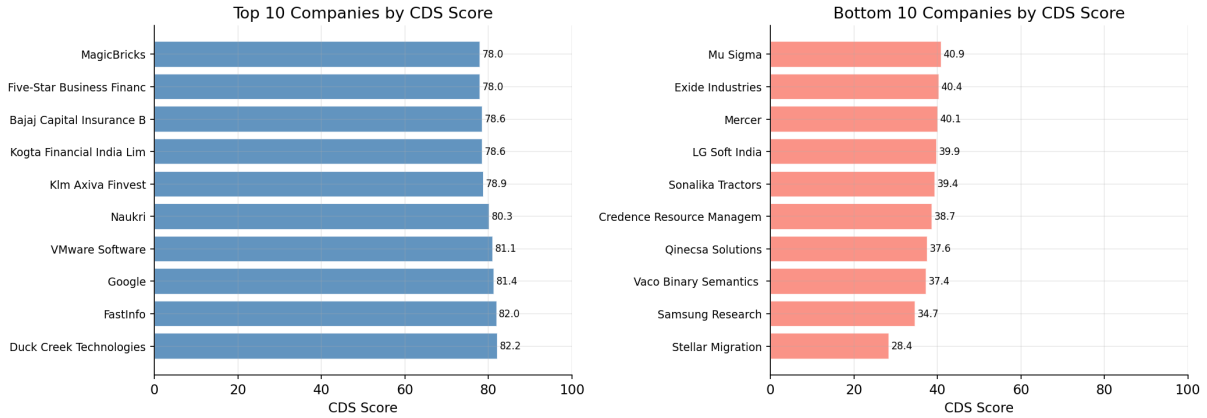


Figure 11: Left: Top 10 companies by predicted CDS score. SaaS and Internet companies dominate, consistent with their engineering-first cultures. Right: Bottom 10 companies, predominantly outsourcing and manufacturing firms where ownership and technical growth opportunities are lower.

Table 5: Top 10 companies by predicted CDS score

Rank	Company	CDS Score	Sector
1	Duck Creek Technologies	82.2	SaaS / Software
2	FastInfo	82.0	Internet / E-Commerce
3	VMware Software	81.4	SaaS / Software
4	Google	80.7	SaaS / Software
5	Naukri	80.7	Internet / E-Commerce
6	MagicBricks	78.6	Internet / E-Commerce
7	Uber	78.5	Internet / E-Commerce
8	SAP	77.3	SaaS / Software
9	Amazon	77.1	Internet / E-Commerce
10	Cummins	76.8	Engineering

9 Discussion and Limitations

9.1 What Works Well

The CDM produces interpretable, differentiated scores. The ranking of Google, SAP, VMware, and product-first companies near the top is consistent with industry reputation. The model achieves $MAE = 1.20$ points on a 0–100 scale, meaning predictions are within about 1 point on average — sufficient for ranking and tier classification.

9.2 Limitations

Weakly supervised labels. The five DNA dimension labels are rule-based proxies derived from employee tags and structural signals. They are not validated against ground-truth developer surveys. Future work should collect labelled responses from 500+ developers.

Empty sub-score columns. The JavaScript-rendered score elements (salary score, job security score, etc.) could not be scraped reliably. These would have provided richer directly-observed signals. A future version using browser automation with sufficient wait time could recover these.

Partial IID violation. Companies in the same sector share vocabulary, creating intra-class correlations. Sector-stratified k-fold CV would give a more conservative accuracy estimate.

10 Conclusion

This project presented the Company DNA Matrix, a machine learning system that scores 1,302 Indian companies across five developer-relevant dimensions. The system scrapes, cleans, and extracts features from AmbitionBox data, constructs independent labels to avoid data leakage, and trains five regression models for comparison.

XGBoost achieves the best performance with Test MAE = 1.20/100 and CV $R^2 = 0.969 \pm 0.008$, outperforming linear baselines because non-linear interactions between TF-IDF features and sector signals cannot be modelled by a dot product. K-Means clustering with $k = 4$ groups companies into four identifiable archetypes, enabling developer-company similarity search.

The result is an interpretable, reproducible ranking system grounded in the developer-centric framework of the original project proposal.

References

- [1] C.J. Hutto and E. Gilbert. *VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text*. Proceedings of the 8th AAAI Conference on Weblogs and Social Media, 2014.
- [2] A. Mukherjee and B. Liu. *Aspect Extraction through Semi-Supervised Modelling from Employee Reviews*. ACL Workshop on Sentiment Analysis, 2019.
- [3] K. Sparck Jones. *A Statistical Interpretation of Term Specificity and its Application in Retrieval*. Journal of Documentation, 28(1):11–21, 1972.

- [4] T. Chen and C. Guestrin. *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD, 2016.
- [5] S.P. Lloyd. *Least Squares Quantization in PCM*. IEEE Transactions on Information Theory, 28(2):129–137, 1982.
- [6] F. Pedregosa et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12:2825–2830, 2011.
- [7] A.E. Hoerl and R.W. Kennard. *Ridge Regression: Biased Estimation for Nonorthogonal Problems*. Technometrics, 12(1):55–67, 1970.
- [8] V.N. Vapnik. *The Nature of Statistical Learning Theory*. Springer, 1995.